

数据库常见面试 Q&A（分级）

初级

Q1: 数据库事务的 ACID 是什么?

A: 原子性、一致性、隔离性、持久性。

Q2: 隔离级别有哪些?

A: 读未提交、读已提交、可重复读、可串行化。

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
BEGIN TRAN;  
-- your sql  
COMMIT;
```

Q3: 脏读、不可重复读、幻读的区别?

A: 脏读是读到未提交数据; 不可重复读是同一行两次读取结果不同; 幻读是同条件两次查询行数不同。

```
-- 幻读演示：同条件两次查询行数变化  
SELECT COUNT(*) FROM Orders WHERE Status = 'Pending';
```

Q4: 索引的作用是什么?

A: 通过有序结构减少全表扫描, 提高查询性能。

```
CREATE INDEX IX_Orders_UserId_CreatedAt  
ON Orders(UserId, CreatedAt DESC);
```

Q5: 覆盖索引是什么?

A: 索引包含查询所需全部列, 避免回表。

```
-- 查询可被覆盖索引命中  
SELECT UserId, Status, Amount, CreatedAt  
FROM Orders  
WHERE UserId = @userId AND Status = 1;
```

中级

Q6: 索引失效常见原因?

A: 函数/计算、前导模糊查询、类型隐式转换、复合索引未使用最左前缀。

```
-- 函数导致索引失效  
SELECT * FROM Users WHERE YEAR(CreatedAt) = 2024;
```

```
-- 前导模糊导致索引失效  
SELECT * FROM Users WHERE Name LIKE '%hao';
```

Q7: 如何定位慢 SQL?

A: 看慢查询日志、执行计划、IO 与 CPU 消耗, 重点关注全表扫描与回表。

```
-- MySQL 执行计划
EXPLAIN SELECT * FROM Orders WHERE UserId = 1;
```

```
-- SQL Server 查看 IO 与耗时
SET STATISTICS IO, TIME ON;
```

Q8: 常见 SQL 优化策略?

A: 合理建索引、避免 SELECT *、拆分复杂查询、减少回表、用覆盖索引。

```
-- 分页优化：基于游标
SELECT * FROM Orders
WHERE Id > @lastId
ORDER BY Id
LIMIT 50;
```

```
-- 覆盖索引示例
CREATE INDEX IX_Orders_Cover
ON Orders(UserId, Status)
INCLUDE (Amount, CreatedAt);
```

Q9: 行锁与表锁区别?

A: 行锁粒度小并发高；表锁粒度大但开销小。

```
-- MySQL 行锁（需要命中索引）
SELECT * FROM Orders WHERE Id = 1 FOR UPDATE;
```

```
-- SQL Server 行级意向锁示例
SELECT * FROM Orders WITH (UPDLOCK, ROWLOCK)
WHERE Id = @id;
```

Q10: 乐观锁与悲观锁区别?

A: 乐观锁用版本号/时间戳控制并发；悲观锁直接加锁阻塞并发。

```
// EF Core 并发控制示例 (RowVersion)
public class Product
{
    public int Id { get; set; }
    public byte[] RowVersion { get; set; }
}
```

```
-- 乐观锁（版本号）
UPDATE Products
SET Stock = Stock - 1, Version = Version + 1
WHERE Id = @id AND Version = @version;
```

高级

Q17: 如果有上亿条数据，需要快速查找有什么方法?

A: 优先明确查询场景与维度，再选择组合方案：合理索引（复合/覆盖/列存）、分区表按时间或主键切分、冷

热分离、读写分离、缓存热点结果、预计算/物化视图；当单库瓶颈明显时采用分库分表或引入搜索引擎（如ES）做检索加速。

```
-- 分区示例（按月）
CREATE PARTITION FUNCTION pf_OrderDate (DATE)
AS RANGE RIGHT FOR VALUES ('2024-01-01', '2024-02-01', '2024-03-01');
```

```
-- 复合索引 + 覆盖索引
CREATE INDEX IX_Orders_UserId_Status
ON Orders(UserId, Status)
INCLUDE (Amount, CreatedAt);
```

Q18：百万级分页如何避免深分页性能问题？

A：避免 OFFSET 深度分页，改用基于游标/主键的“延续查询”，或使用分区+时间范围缩小扫描；必要时结合缓存热门页。

```
-- 游标分页（延续查询）
SELECT * FROM Orders
WHERE Id > @lastId
ORDER BY Id
LIMIT 50;
```

Q19：日志或流水表增长很快，如何做归档与查询兼顾？

A：按时间分区表或分表，冷热数据分离；历史数据归档到历史库或对象存储；报表查询用物化视图/汇总表。

```
-- 将历史数据归档
INSERT INTO Orders_History SELECT * FROM Orders WHERE CreatedAt < '2023-01-01';
DELETE FROM Orders WHERE CreatedAt < '2023-01-01';
```

Q20：如何设计电商订单的多维查询（用户、状态、时间）？

A：基于高频过滤条件建立复合索引，遵循最左前缀；拆分读写模型，核心表保持范式，查询表可反范式冗余；热点报表用缓存或物化视图。

```
CREATE INDEX IX_Orders_User_Status_Date
ON Orders(UserId, Status, CreatedAt DESC);
```

Q21：如何在海量数据下做模糊搜索？

A：数据库端避免前导模糊；可通过倒排索引的搜索引擎（ES）实现，或维护搜索关键字表；必要时增加前缀索引或全文索引。

```
-- MySQL 全文索引（示例）
ALTER TABLE Articles ADD FULLTEXT INDEX IX_FT_TitleBody(Title, Body);
```

Q22：如何提升统计报表（聚合/分组）性能？

A：避免实时全表聚合，使用汇总表、物化视图或列存索引；按时间维度分区减少扫描；定时批处理预计算。

```
-- 预聚合示例
CREATE TABLE DailySales (
    SaleDate DATE PRIMARY KEY,
    TotalAmount DECIMAL(18,2)
);
```

Q23: 如何处理“热行”导致的锁竞争?

A: 减少热点更新, 拆分热字段、使用异步累加、分桶计数; 对热点行使用乐观锁或行级锁, 缩短事务时间。

```
-- 分桶计数示例: 按哈希分散热点
UPDATE Counters
SET Value = Value + 1
WHERE BucketId = @bucketId;
```

Q11: 第一范式 (1NF) 是什么?

A: 字段不可再分, 列值必须是原子值。

```
-- 不符合 1NF (地址拆分)
-- Address = "Beijing, Haidian"
```

Q12: 第二范式 (2NF) 是什么?

A: 满足 1NF, 非主属性完全依赖主键, 消除部分依赖。

```
-- 复合主键示例
CREATE TABLE OrderItems (
    OrderId BIGINT,
    ProductId BIGINT,
    Quantity INT,
    PRIMARY KEY (OrderId, ProductId)
);
```

Q13: 第三范式 (3NF) 是什么?

A: 满足 2NF, 消除传递依赖, 非主属性只依赖主键。

```
-- 传递依赖拆分
-- Orders(UserId, UserName) -> Users(Id, Name)
```

Q14: BCNF 是什么?

A: 所有函数依赖的决定因素必须是候选键。

示例: 若 $A \rightarrow B$ 且 A 不是候选键, 则违反 BCNF

Q15: 什么时候反范式?

A: 读多写少、复杂查询成本高时引入冗余, 但需一致性维护机制。

```
-- 冗余字段同步 (触发器示例)
CREATE TRIGGER trg_orders_user
ON Users AFTER UPDATE
AS
UPDATE Orders
SET UserName = i.Name
FROM inserted i
WHERE Orders.UserId = i.Id;
```

```
-- 反范式示例: 订单表冗余用户姓名
ALTER TABLE Orders ADD UserName NVARCHAR(50);
```

Q16: 分布式事务常见方案?

A: 2PC、TCC、Saga、消息最终一致性。

```
-- 消息表 + 状态字段
ALTER TABLE Outbox ADD Status INT NOT NULL DEFAULT 0;
```

```
-- Outbox 示例表
CREATE TABLE Outbox (
    Id BIGINT PRIMARY KEY,
    AggregateId BIGINT NOT NULL,
    EventType NVARCHAR(50) NOT NULL,
    Payload NVARCHAR(MAX) NOT NULL,
    CreatedAt DATETIME2 NOT NULL
);
```